

第23回 実践：Web ゲームプログラミング2（キー入力）

実習時のフォルダ構成

Web_名前 の中に 23 フォルダを作る。第 22 回の `sample` フォルダと `exercise` フォルダを、23 の中にコピーして、続きから作る。

```
Web_名前
├── 23
│   ├── sample          ← 第 22 回の sample をコピーする
│   │   ├── index.html
│   │   └── main.js
│   └── exercise        ← 第 22 回の exercise をコピーする
│       ├── index.html
│       └── main.js
```

- `sample`：講義で使うサンプル（講義内容のコードは、`sample/main.js` に書き足す）
- `exercise`：自分のシューティングゲーム（演習 1～2、発展）
- `index.html` は、第 22 回のままで変更しない。

講義内容

`sample/main.js` に書き足す。書いたら `sample/index.html` をブラウザで開いて確認する。キー入力を確認するときは、ブラウザの画面を一度クリックしてから、キーを押す。

1. 前回の復習

- ゲームループ：`gameLoop` の中で「`update`（更新）→ `draw`（描画）→ `requestAnimationFrame(gameLoop)`」を繰り返す。約 60 回／秒。
- 毎フレーム、背景を塗ってから描き直す。

今回は、キーボードの ← → キーで、自機を左右に動かす。

2. 今回の `main.js` の並び

今回は「キー入力」の部分を追加し、`updatePlayer` を書き換える。

```
// ===== 準備 =====
// ===== ゲームの状態 =====
// ===== 自機 =====
// ===== キー入力 =====    ← 今回追加する
// ===== 更新 =====        ← updatePlayer を書き換える
// ===== 描画 =====
// ===== ゲームループ =====
```

3. イベント

「キーが押された」「キーが離された」のように、ブラウザで起きた出来事を「イベント」という。イベントが起きたときに実行する処理は、`addEventListener` で登録する。

```
document.addEventListener("イベントの種類", function (event) {  
    イベントが起きたときに実行する処理  
});
```

イベントの種類	起きるとき
"keydown"	キーが押されたとき
"keyup"	キーが離されたとき

- `function (event) { }` は、名前のない関数。その場で関数を作って、`addEventListener` に渡している。
- イベントが起きるたびに、ブラウザがこの関数を呼び出す。そのとき、引数 `event` に、イベントの情報（どのキーか、など）が入る。
- `event.key` で、押されたキーの名前が分かる。

キー	<code>event.key</code> の値
← → ↑ ↓	"ArrowLeft" "ArrowRight" "ArrowUp" "ArrowDown"
スペース	" "（空白 1 文字）
Enter	"Enter"
A	"a"（Shift を押しながらだと "A"）

4. キーの状態を変数に記録する

「自機」の部分の下に、「キー入力」の部分を追加する。

```
// ===== キー入力 =====
let isLeftPressed = false;
let isRightPressed = false;

document.addEventListener("keydown", function (event) {
  console.log(event.key); // 押したキーの名前を確認する
  if (event.key === "ArrowLeft") {
    isLeftPressed = true;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = true;
  }
});

document.addEventListener("keyup", function (event) {
  if (event.key === "ArrowLeft") {
    isLeftPressed = false;
  }
  if (event.key === "ArrowRight") {
    isRightPressed = false;
  }
});
```

- `isLeftPressed` : ← キーが押されている間は `true`、離すと `false` になる。
- キーを押したときに、自機を直接動かすのではなく、「押されているかどうか」だけを変数に記録する。実際に動かすのは、ゲームループの `updatePlayer` で行う。
- こうすると、キーを押している間、毎フレームなめらかに動き続ける。
- `console.log(event.key);` で、押したキーの名前がコンソールに表示される。いろいろなキーを押して確認する（この行は、次回、書き換えるときに消す）。

5. updatePlayer を書き換える

「更新」の部分の `updatePlayer` を、次のように書き換える。

```
function updatePlayer() {
  if (isLeftPressed) {
    player.x -= player.speed;
  }
  if (isRightPressed) {
    player.x += player.speed;
  }
  // 画面の端で止める
  if (player.x < 0) {
    player.x = 0;
  }
  if (player.x + player.w > canvas.width) {
    player.x = canvas.width - player.w;
  }
}
```

- ← が押されていれば左へ、→ が押されていれば右へ、`speed` だけ動かす。

- 第 18 回で学んだ「画面の端で止める」処理。右端は、自機の右端 (`player.x + player.w`) で比べる。

演習

`exercise/main.js` (自分のシューティングゲーム) に書く。

演習 1 キーで自機を左右に動かす

- 講義と同じように「キー入力」の部分を追加し、`updatePlayer` を書き換えて、`←` `→` キーで自機が左右に動くようにする。
- 第 22 回の演習 2 で左へ動き続けるようにした `updatePlayer` は、ここで書き換える。
- 左右の端で、自機が止まることを確認する。

演習 2 別のキーでも動かす

- `console.log(event.key)` で、`A` キーと `D` キーの名前を確認する。
- `A` キーでも左へ、`D` キーでも右へ動くようにする。(ヒント: `keydown` と `keyup` の両方の条件を、`||` を使って書き換える。)

発展 (任意)

- `↑` `↓` キーで、自機が上下にも動くようにする。上下の端でも止まるようにする。